

IFN647 Week 12 Review Questions

Solutions

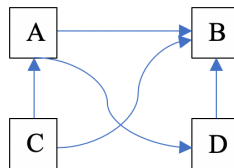
Question 1 (PageRank)

A Web graph $G = (P, L)$ consists of Web pages (vertices) and links (edges). The PageRank procedure takes a Web graph G as input and then outputs the better PageRank estimate PR using the following equation

$$PR(u) = \frac{\lambda}{N} + (1 - \lambda) \cdot \sum_{v \in B_u} \frac{PR(v)}{L_v}$$

where B_u is the set of pages that point to u , and L_v is the number of outgoing links from page v .

The following figure $G = (P, L)$ is a Web graph, where $P = \{A, B, C, D\}$ and $L = \{(A, B), (A, D), (C, A), (D, B), (C, B)\}$.



Assume the initial estimate of each Web page is equally, i.e., $PR(A) = PR(B) = PR(C) = PR(D) = 0.25$; and $\lambda = 0.15$. Calculate the PageRank estimate (PR value) of each Web page after running the first iteration of procedure $\text{PageRank}(G)$.

Solution:

The first iteration:

$$L_C = |\{A, B\}| = 2, \quad L_A = |\{B, D\}| = 2, \quad L_D = |\{B\}| = 1, \quad L_B = |\{\}| = 0,$$

For page A: $B_A = \{C\}$

$$\begin{aligned} R(A) &= 0.15/4 + (1-0.15) * (R(C) / L_C) \\ &= 0.0375 + 0.85 * (0.25/2) = 0.14375 \end{aligned}$$

For page B: $B_B = \{A, C, D\}$

$$R(B) = 0.15/4 + (1-0.15) * (R(C) / 2 + R(A) / 2 + R(D) / 1)$$

$$= 0.0375 + 0.85 * (0.25/2 + 0.25/2 + 0.25) = 0.4625$$

For page C: $B_c = \{\}$

$$R(C) = 0.15/4 + (1-0.15)*0 = 0.0375$$

For page D: $B_D = \{A\}$

$$\begin{aligned} R(D) &= 0.15/4 + (1-0.15) * (R(A)/2) \\ &= 0.0375 + 0.85 * (0.25/2) = 0.14375 \end{aligned}$$

Question 2 (Attention to BERT)

In Week 3, we discussed BERT-base word embeddings, where a pre-trained tokenizer and model can be loaded as follows:

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')
```

Week 3 Review Question 7 demonstrated how to use a standard pre-trained BERT-base model to encode two input texts and generate their corresponding token-level (word) embeddings. The code covered both the initial encoding process and the final step of obtaining word embeddings.

In this week, you will extend this code to better understand how the `attention_mask` is used to obtain encoder outputs. Your implementation may include the following steps:

1. Tokenize an input text
2. Generate initial embeddings (i.e., encoder inputs), and optionally print the representation of the first token
3. Pass the initial embeddings to the BERT encoder using the `attention_mask` to obtain the encoder outputs
4. Print the representation of the first token again to observe the differences

Solution:

```
# (1) Tokenize an input text
inputs = tokenizer(text1,
                    padding=True,
                    truncation=True,
                    return_tensors="pt",
                    add_special_tokens=True)

# Get token embeddings layer from BERT
embedding_layer = model.embeddings

# (2) Generate initial embeddings
embeddings = embedding_layer(
    input_ids=inputs["input_ids"],
    token_type_ids=inputs["token_type_ids"] )

print("\nEmbedding Shape:")
print(embeddings.shape)
print("\nFirst Token Embedding:")
print(embeddings[0, 0, :20])

Embedding Shape:
torch.Size([1, 10, 768])

First Token Embedding:
tensor([ 0.1686, -0.2858, -0.3261, -0.1122,  0.0343, -0.2689, -0.0302, -0.0390,
         0.0157, -0.2828,  0.1436,  0.0426, -0.2203, -0.0023, -0.5525,  0.1092,
        -0.0211, -0.0151,  0.0724, -0.3034], grad_fn=<SliceBackward0>)

# Notes that: when we input text into BERT, all sequences must have the same length.
# So shorter sentences are padded with special tokens (usually set padding=True in "tokenizer")

# (3) Pass Embeddings into BERT Encoder

encoder_outputs = model.encoder(
    embeddings,
    attention_mask = inputs["attention_mask"].bool()
)

# the attention_mask tells the encoder: Which tokens are real words and which tokens are just padding.
# It is usually a binary mask: 1 - attend to this token, and 0 - ignore this token (usually padding)

# (4) print out the final contextual representations
last_hidden_state = encoder_outputs.last_hidden_state
print("\nEncoder Output Shape:")
print(last_hidden_state.shape)
# Example contextualized vector
print("\nFirst Token Contextualized Representation:")
print(last_hidden_state[0, 0, :20])

Encoder Output Shape:
torch.Size([1, 10, 768])

First Token Contextualized Representation:
tensor([-0.5284, -0.4093, -0.4727, -0.1305, -0.1049,  0.0786, -0.1079,  0.4094,
        -0.6169, -0.2246, -0.0838,  0.1648, -0.0798, -0.3042, -0.2275, -0.0727,
        -0.5986,  0.4513,  0.2165,  0.0188], grad_fn=<SliceBackward0>)
```

Question 3 (Application of unsupervised contrastive learning)

How can pseudo-relevant feedback techniques be effectively integrated with contrastive learning to identify and construct potential positive samples in a learning framework? Building on the concepts covered in this unit, can you design a clear, step-by-step pipeline that demonstrates how pseudo-correlations are generated, refined, and incorporated into contrastive training?

Additionally, can you illustrate this process with a concrete example for a given query Q , showing how candidate samples are selected, evaluated, and used to form positive pairs?

Solution:

Inputs: a query Q , a baseline IR model (e.g., **BM25**) and Retrieved top- k documents

Output: An encoder that increases the similarity between the query and positive samples, $\text{sim}(Q, d^+)$, while decreasing the similarity between the query and negative samples, $\text{sim}(Q, d^-)$.

#Assume: Top_ k _docs = **pseudo positives**, and Others = **negatives**

Step 1: Build Training Pairs

Positive pairs = (Q, d^+) where $d^+ \in \text{Top}_k\text{-docs}$

Step 2: Select **Negative pairs**

Random documents OR low-ranked documents

Hard negatives: documents ranked just below top- k

Step 3: Train a contrastive model

Example:

Query Q : “deep learning for NLP”

Assume retrieved top-3:

Doc1: transformer models #positive

Doc2: neural networks NLP #positive

Doc3: deep learning overview #positive

Negatives:

Doc10: cooking recipes

Doc20: sports news

Training pairs:

$(Q, \text{Doc1}), (Q, \text{Doc2}), (Q, \text{Doc3}) \rightarrow \text{positive}$

$(Q, \text{Doc10}), (Q, \text{Doc20}) \rightarrow \text{negative}$

Assume: Train encoder using contrastive loss (e.g., InfoNCE) and return an encoder $\text{Encoder}()$.

Encode the query: $Q_{vec} = \text{Encoder}(Q)$

Encode all documents: $D_i^{vec} = \text{Encoder}(D_i)$

Compute Similarity: $\text{sim}(Q, D_i) = Q_{vec} \cdot D_i^{vec}$

Sort documents by similarity score and select top- k documents again.

Question 4 (Text Contrastive Learning) – Optional

Read and test the provided code for this question. The code uses a small dataset of text sentences and defines functions to generate text pairs using a simple data augmentation strategy. It then defines an encoding function to convert these texts into vector representations, as well as a contrastive loss function.

The code further trains a contrastive learning model over several epochs and ultimately produces a trained embedding model capable of encoding any input text sentence into a meaningful vector representation.

Please ensure that you understand the procedure for computing the contrastive loss defined in the function `contrastive_loss(z1, z2, temperature=0.5)`. Then, express this procedure as an algorithm in plain English.

Solution:

Algorithm (contrastive loss) # It is a SimCLR-style contrastive loss (InfoNCE variant).

Inputs: `z1` (embeddings from `x1`), `z2` (embeddings from `x2`), and temperature $\tau=0.5$,
`x1` is a batch of sentences and `x2` contains their corresponding positive augmentations

Output: Mean contrastive loss over all samples in the batch

Method:

- (1) Both `z1` and `z2` scaled to unit vectors so that dot product becomes cosine similarity.
- (2) Merge `z1` and `z2` to `z`, i.e., $z = \begin{bmatrix} z1_1, & z1_2, & \dots, & z1_N, \\ z2_1, & z2_2, & \dots, & z2_N \end{bmatrix}$
- (3) Compute similarity matrix - $\text{sim}(z_i, z_j) / \tau$ # using `sim = torch.matmul(z, z.T) / temperature`
- (4) Build labels for positives `[0, 1, ..., N-1, 0, 1, ..., N-1]`
Each sentence appears twice (two views: itself and augmented sentence).
- (5) Remove self-similarity by set $\text{sim}(z_i, z_i) = -\infty$ # a very large negative value
- (6) Extract positive pairs; e.g., positive of `z1[i]` is `z2[i]`, positive of `z2[i]` is `z1[i]`. Thus, positive logits become $[\text{sim}(z1_i, z2_i), \text{sim}(z2_i, z1_i)]$.
- (7) Compute contrastive loss
$$l_i = -\text{sim}(i, i^+) + \log \left(\sum_j \exp(\text{sim}(i, j)) / \tau \right)$$

This is equivalent to the standard formulation used in the lecture notes.
The difference is only algebraic rewriting of the log-softmax form:
 $-\text{sim}(i, i^+) \text{ vs. } -\log(\exp(\text{sim}(i, i^+) / \tau))$
- (8) Return the averaging loss.